

Brain MRI Tumor Classification

Model Development, Training & Evaluation Report

Dataset: BRISC 2025 — Classification Task

Scope: Deep learning architecture, training, and evaluation

Contents

- 1. Project Scope and Objective
- 2. Dataset Structure and Data Contract
- 3. Classification Dataset Preprocessing
- 4. Train/Validation Split
- 5. ResNet34 Classification Architecture
- 6. Loss Function and Class Weighting
- 7. Optimization and Training Strategy
- 8. Evaluation Protocol
- 9. Quantitative Results
- 10. Training and validation figures
- 11. Confusion Matrix
- 12. Qualitative Test Predictions
- Appendix. Final Metrics

1. Project Scope and Objective

The classification stage of the BRISC 2025 project is a four-class brain MRI classification task. The final pipeline receives a single-channel grayscale MRI image and predicts one of four diagnostic classes: glioma, meningioma, no_tumor, or pituitary.

Item	Final configuration
Task	Four-class brain MRI image classification
Classes	glioma/meningioma/no_tumor/pituitary
Input	Single-channel grayscale MRI
Working resolution	256 × 256 pixels
Train/validation policy	80/20 stratified split from official training split
Official test set	1,000 samples; kept untouched for final evaluation
Final backbone	ImageNet-pretrained ResNet34
Final checkpoint	best_resnet34_classifier.pth
Primary selection metric	Validation macro F1

2. Dataset Structure and Data Contract

The classification notebook expects an ImageFolder-style directory structure: a train folder and a test folder, each containing one subfolder per class. The four class names discovered by the dataset are sorted and mapped to integer indices. The final test run contained 1,000 images with the following class support.

Test class	Support
glioma	254
meningioma	306
no_tumor	140
pituitary	300
Total	1,000

The project notebook also contains dataset auditing utilities for file extensions, image validity, size distribution, image mode distribution, class distribution, and small-image outliers. These audit cells are intended to establish data integrity before training.

3. Classification Dataset Preprocessing

The final Dataset implementation follows an ImageFolder-style directory and applies deterministic preprocessing to validation/test samples and randomized augmentation to training samples. Images are converted to grayscale, resized to 256 × 256, converted to tensors, and normalized with mean=0.5 and standard deviation=0.5.

Stage	Implementation
Mode	Grayscale L; one channel
Resize	Bilinear interpolation to 256 × 256
Train augmentation	Random horizontal flip; random rotation in [-15°, +15°]; random brightness in [0.8, 1.2]
Validation/Test augmentation	Disabled
Tensor conversion	torchvision.transforms.functional.to_tensor
Normalization	mean=[0.5], std=[0.5]
Label type	torch.long class index

```

image = image.convert("L")
image = TF.resize(image, (256, 256),
                    interpolation=TF.InterpolationMode.BILINEAR)
image = TF.to_tensor(image)
image = TF.normalize(image, mean=[0.5], std=[0.5])

```

4. Train/Validation Split

The official training split was divided into training and validation subsets using `train_test_split` with `test_size=0.20`, `random_state=42`, `shuffle=True`, and `stratify=all_labels`. Training augmentation is enabled only on the training subset; validation augmentation is disabled. The official test directory is never included in this split.

```

train_indices, val_indices = train_test_split(
    all_indices,
    test_size=0.20,
    random_state=42,
    shuffle=True,
    stratify=all_labels
)

```

Subset	Construction	Augmentation
Train	Subset(train_dataset_full, train_indices)	Enabled
Validation	Subset(val_dataset_full, val_indices)	Disabled
Test	Independent TEST_DATASET_PATH	Disabled

The split is stratified at the image/class-label level. A separate duplicate sanity check was also performed during project validation and did not reveal a train/test duplication issue.

5. ResNet34 Classification Architecture

The final classifier is `ResNet34Classifier`, built from the torchvision ImageNet-pretrained ResNet34 backbone. The original RGB stem is adapted to the one-channel MRI input by replacing `conv1` with a one-channel convolution and initializing it from the mean of the three pretrained RGB filters. The original fully connected layer is replaced with a four-class linear classifier.

Component	Implementation
Backbone	ResNet34, ImageNet pretrained
Input channels	1
Stem adaptation	Pretrained RGB conv1 weights averaged across input-channel dimension
Feature extractor	ResNet34 convolutional stages
Classifier head	Linear(512, 4)
Trainable parameters	All model parameters are trainable
<pre> original_conv = backbone.conv1 new_conv = nn.Conv2d(in_channels=1, out_channels=original_conv.out_channels, kernel_size=original_conv.kernel_size, stride=original_conv.stride, padding=original_conv.padding, bias=False) with torch.no_grad(): new_conv.weight.copy_(original_conv.weight.mean(dim=1, keepdim=True)) backbone.conv1 = new_conv backbone.fc = nn.Linear(512, 4) </pre>	

The forward-pass check verified that an input batch with shape [B, 1, 256, 256] produces an output tensor with shape [B, 4].

6. Loss Function and Class Weighting

The final classifier was trained with weighted cross-entropy. Class weights were computed from the training labels using the standard balanced weighting formula $N/(K \times n_c)$, then supplied to CrossEntropyLoss. This explicitly increases the contribution of classes with fewer training examples.

```

class_weights = len(train_labels) / (NUM_CLASSES * class_counts)
class_weights = torch.tensor(
    class_weights, dtype=torch.float32, device=device
)
criterion = nn.CrossEntropyLoss(weight=class_weights)

```

7. Optimization and Training Strategy

Setting	Value
Epoch limit	45
Batch size	16
Learning rate	1e-4

Setting	Value
Weight decay	1e-5
Optimizer	AdamW
Scheduler	ReduceLROnPlateau
Scheduler mode	max
Scheduler factor	0.5
Scheduler patience	3
Early stopping patience	7 epochs
Checkpoint criterion	Best validation macro F1

During each epoch, the model is trained on the training loader, evaluated on the validation loader, and the validation macro F1 is used for scheduler stepping and best-checkpoint selection. When the validation F1 improves, the state_dict is saved to best_resnet34_classifier.pth and restored after training.

```
if val_f1 > best_val_f1:
    best_val_f1 = val_f1
    best_model_state = copy.deepcopy(model.state_dict())
    torch.save(best_model_state, "best_resnet34_classifier.pth")
```

8. Evaluation Protocol

The final evaluation reports overall accuracy together with macro-averaged precision, recall, and F1. A confusion matrix and full per-class classification report are generated from the same test predictions. The test split contains 1,000 images and is evaluated only after the training stage and best-checkpoint restoration.

Metric	Definition/reporting style
Accuracy	Fraction of correctly classified test samples
Precision	Macro-average across the four classes
Recall	Macro-average across the four classes
F1	Macro-average across the four classes
Per-class report	Precision, recall, F1, and support for each class
Confusion matrix	Counts of true-vs-predicted classes

9. Quantitative Results

Metric	Final test result
Accuracy	0.9930
Macro Precision	0.9928
Macro Recall	0.9941

Metric	Final test result
Macro F1	0.9934

9.1 Per-class results

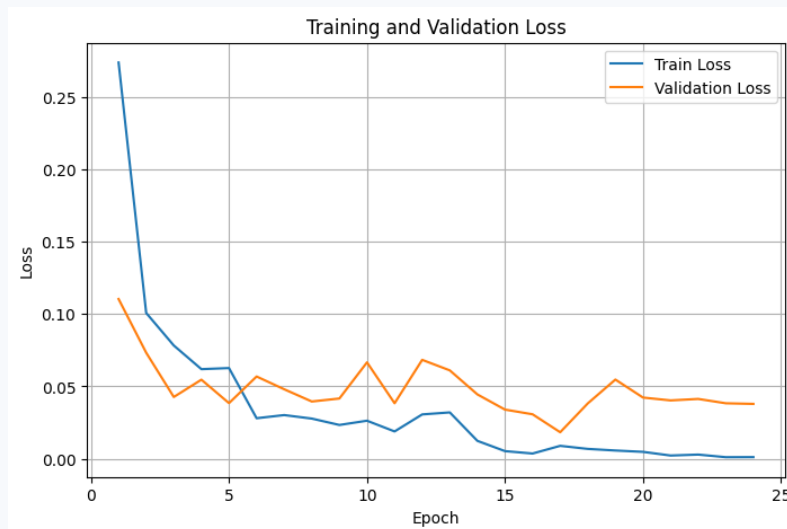
Class	Precision	Recall	F1	Support
Glioma	0.9883	0.9961	0.9922	254
Meningioma	0.9902	0.9869	0.9885	306
No tumor	0.9929	1.0000	0.9964	140
Pituitary	1.0000	0.9933	0.9967	300

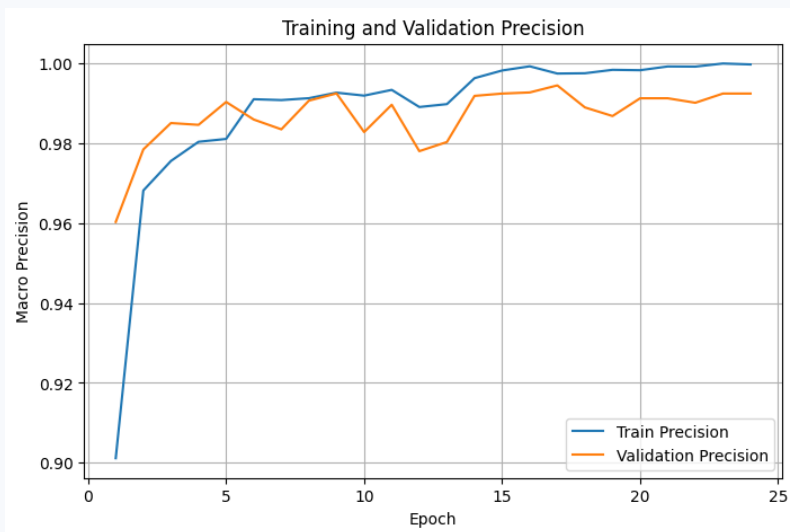
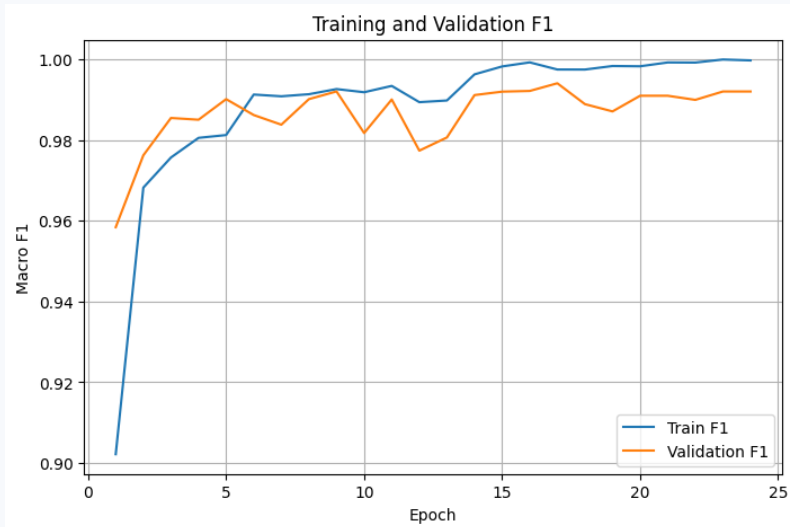
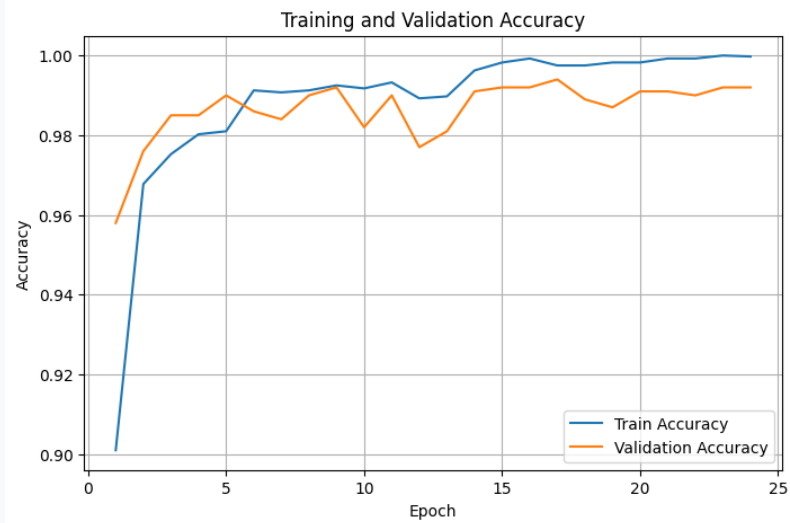
The observed test performance is consistently high across all four classes. The perfect class-level recall for no_tumor and perfect class-level precision for pituitary are properties of this specific 1,000-sample test result and are not, by themselves, evidence of a metric calculation error.

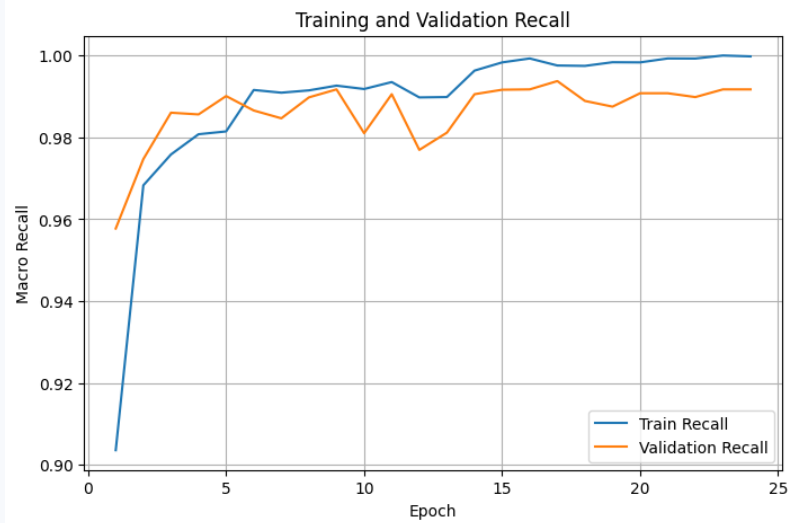
9.2 Final result

Final recorded performance: 99.30% test accuracy and 99.34% macro F1. These values were obtained with the pretrained ResNet34 classifier, grayscale 256×256 inputs, training-only augmentation, weighted cross-entropy, AdamW optimization, validation-F1 checkpoint selection, and the untouched 1,000-image test split.

10. Training and validation figures

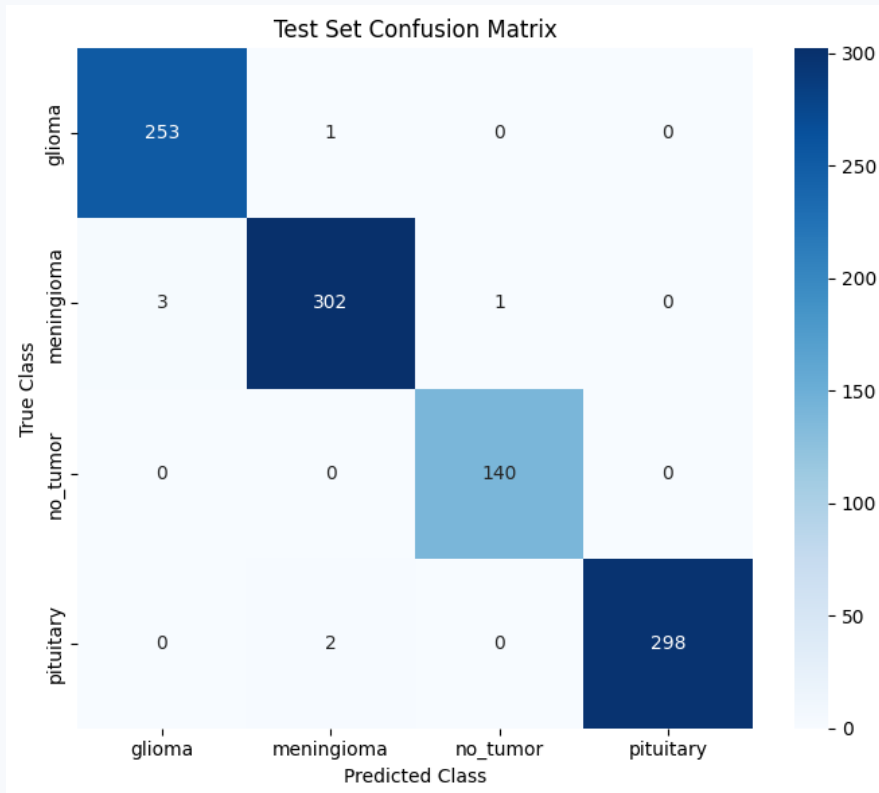




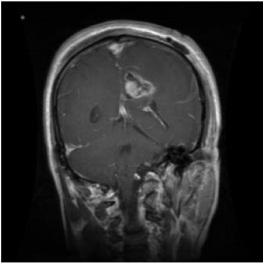
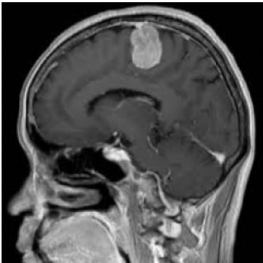
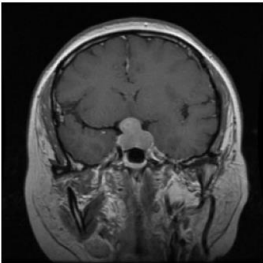
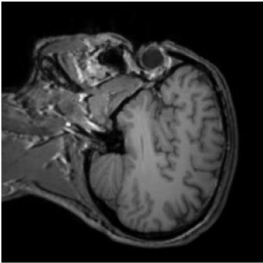


11. Confusion Matrix

The final confusion matrix is generated from all_targets and all_predictions on the test set. The per-class report is generated with zero_division=0 and uses the same class order discovered by the training dataset.



12. Qualitative Test Predictions

Sample	Original MRI	True Class	Predicted Class
1		Glioma	Glioma
2		Meningioma	Meningioma
3		Pituitary	Pituitary
4		No tumor	No tumor

Appendix. Final Metrics

Metric	Final value
Accuracy	0.9930
Macro Precision	0.9928

Metric	Final value
Macro Recall	0.9941
Macro F1	0.9934
Glioma F1	0.9922
Meningioma F1	0.9885
No tumor F1	0.9964
Pituitary F1	0.9967